Day 26: Lakehouse Federation

Data Move Kiye Bina Query Karo

Core idea: Connect an external system, expose it through a Foreign Catalog in Unity Catalog, and query the data where it already lives.

60-90 Second Reel Script

Visual / Action / B-Roll	Spoken Script
Fast hook: SQL Server, PostgreSQL, Snowflake + Databricks. Text: "Data Copy Kiye Bina Query?"	Har baar data Databricks mein ingest karna zaroori nahi hai. Agar aapka data SQL Server, PostgreSQL, Snowflake ya kisi external database mein already pada hai, toh Databricks us data ko wahi se directly query kar sakta hai using Lakehouse Federation.
External Database → Databricks, with a large NO COPY symbol.	Normally analytics ke liye hum data pehle Databricks mein ingest karte hain. Lekin Federation mein data physically lakehouse mein copy nahi hota. Databricks external system se connect karta hai aur data ko in place query karta hai.
Step 1 — Connection. Show host URL + credentials + database.	Iske setup mein pehla component hota hai Connection. Connection basically Databricks ko batata hai ki external database kahan hai aur usse authenticate kaise karna hai — jaise host, port aur credentials.
Step 2 — Foreign Catalog. External DB → Foreign Catalog → Unity Catalog.	Uske baad hum create karte hain Foreign Catalog. Yeh external database ko Unity Catalog ke andar represent karta hai, jisse uske schemas aur tables Databricks mein normal catalog ki tarah visible ho jaate hain.
Show query: SELECT * FROM foreign_catalog.sales.orders	Phir aap simply query kar sakte ho — foreign_catalog.schema.table — almost waise hi jaise kisi normal Unity Catalog table ko query karte ho.
Show WHERE year = 2026 pushed to source. Only filtered rows return. Text: Predicate Pushdown.	Aur smart part ye hai ki Databricks supported filters ko remote database tak push down karne ki try karta hai. For example, agar mujhe sirf 2026 ka data chahiye, toh ideally filter SQL Server par hi execute hoga aur sirf required rows network se Databricks tak aayengi.
Presenter + text: Best For — Ad-hoc / POC / Migration.	Federation especially useful hai ad-hoc reporting, proof of concepts, operational data exploration, ya migration phase mein — jab aap data move kiye bina quickly insights chahte ho.
Warning visual: repeated heavy workloads → ingestion pipeline / Lakeflow Connect.	Lekin Federation ingestion ka replacement nahi hai. Agar data volumes high hain, queries repeatedly chalni hain, ya low-latency analytics chahiye, toh supported sources ke liye data ingest karna better option ho sakta hai.
Core flow: Connection → Foreign Catalog → Query External Data.	So remember: Connection source se connect karta hai, Foreign Catalog us source ko Unity Catalog mein expose karta hai, aur Lakehouse Federation data ko move kiye bina query karne deta hai.
End card: Day 26 complete.	Agar aapko is topic ke advanced detailed notes chahiye, toh niche comment karein DATABRICKS. Aur aise deep tech content ke liye page ko follow karna mat bhoolna.

What Lakehouse Federation Actually Does

Lakehouse Federation lets Databricks query data that remains in external systems. For query federation, Databricks connects to the source, registers it in Unity Catalog, and allows users to query local and foreign relations using familiar catalog.schema.table syntax.

The Two Main Objects

Object	Purpose	Simple Mental Model
Connection	A Unity Catalog securable object that stores the path/endpoint and credentials needed to access the external system.	How Databricks reaches the source.
Foreign Catalog	A catalog in Unity Catalog that represents the external database/catalog and exposes its schemas and tables.	How the external source appears inside Databricks.

End-to-End Flow

External Source → Connection → Foreign Catalog → Unity Catalog Governance → Databricks SQL / Spark

Example

Suppose sales data is still in SQL Server. You create a connection to that SQL Server, create a foreign catalog that mirrors the database, grant Unity Catalog privileges, and then query it:

```
SELECT *
FROM sqlserver_catalog.sales.orders
WHERE order_year = 2026;
```

Predicate Pushdown

Why it matters: Databricks Runtime tries to push supported predicates to the remote database so fewer records need to cross the network. If one predicate cannot be pushed, other compatible predicates joined with AND can still be pushed down.

Useful command: `EXPLAIN FORMATTED` can help you inspect what is being sent to the remote database.

When Federation Is a Good Fit

- Ad-hoc exploration when you do not want to build ingestion first.
- Proofs of concept and discovery work.
- Migration phases where source systems still remain authoritative.
- Operational or reference data that needs to be joined with lakehouse data.
- Situations where copying the data creates unnecessary duplication.

When Ingestion May Be Better

- Large datasets queried repeatedly.

- Heavy joins, aggregations, transformations, or complex analytics.
- Low-latency / predictable performance requirements.
- Workloads that would create too much network traffic or load on the source.
- Cases where you want Delta Lake optimization, change processing, or independent retention.

Important Clarification: Federation ≠ Ingestion

Federation	Ingestion
Data stays in the source system.	Data is copied into the lakehouse.
Query performance depends partly on source and network.	Queries run on data stored in Databricks.
Great for quick access and transitional use cases.	Better for repeated analytics and transformations.
Can push supported work to the remote engine.	Can use Delta/Lakehouse optimizations directly.

2026 Performance Notes

- Predicate pushdown reduces data transferred over the network when filters can be translated to the remote system.
- Databricks documents fetch-size tuning and parallel JDBC reads for supported connectors on newer runtimes.
- Join pushdown is available for several sources; availability and GA/Preview status vary by connector.
- Always test source-system impact: federation can shift compute work back to the operational database.

Visual Summary

Use this collage as a quick Day 26 revision sheet or reel B-roll reference.



Interview-Ready Answer

"Lakehouse Federation lets Databricks query external data sources without first ingesting the data. We create a Unity Catalog Connection to define how Databricks reaches the source, then a Foreign Catalog to expose the external schemas and tables. Supported predicates and, for some connectors, joins can be pushed down to the source to reduce network transfer. Federation is useful for ad-hoc access, POCs, and migration scenarios, but for large repeated analytics workloads, ingestion is often the better architecture."

Quick Revision

- Connection = how Databricks securely reaches the source.
- Foreign Catalog = how the external database appears inside Unity Catalog.
- Federation = query data where it already lives.
- Predicate pushdown = execute supported filters at the source.
- Federation is not automatically better than ingestion; choose based on workload pattern.
- Unity Catalog still controls access to the federated objects.

Official Databricks References

- **Federated queries (Lakehouse Federation):** https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-federated-queries
- **Manage connections for Lakehouse Federation:** https://docs.databricks.com/aws/en/query-federation/connections
- **Lakehouse Federation performance recommendations:** https://docs.databricks.com/aws/en/query-federation/performance-recommendations